Containers

Apptainer / Singularity

Containers provide an isolated environment that supports user applications. In many cases, it is helpful to use a container to obtain the right environment for your applications on HPC clusters, so as to avoid installing too many dependencies.

Containers have great portability and mobility, making it convenient to migrate your applications between different platforms, such as laptops/desktops, cloud platforms, and HPC clusters.

The most well-known container software is Docker, which is designed for laptops/desktops and cloud platforms. On ORCD clusters, we use Apptainer or Singularity instead, which are particularly designed for HPC. Apptainer is extended from Singularity. Both are compatible with Docker.

Note

In the following, the terminology "Singularity" will be used in most cases. The statements hold if the terminologies "Singularity" and "Apptainer" are switched.

Users can use Singularity to support many applications, such as Python, R, C/Fortran packages, and many GUI software. In particular, containers are popular in supporting Python packages for the artificial intelligence (AI) and data science communities, such as PyTorch, TensorFlow, and many others. The Ubuntu operating system (OS) is widely used in the AI community, and it is convenient to install many AI applications in Ubuntu environments. Users can use Singularity to obtain the Ubuntu OS rather than the Rocky 8 OS on the host cluster.

In this document, we will focus on how to use Singularity on ORCD clusters. First, many applications are well-supported in existing Docker images. Search for an image on the internet, in which your target application has already been installed by some developers, then download the image and use it directly. If there is no suitable image for your target application, you can build an image to support it.

> **Note**
>
> An "image" is a file (`.sif` for Singularity/Apptainer) to support a container. Users can launch a containter based on an image.

Run applications with Singularity

Let us start by running an application with Singularity on the cluster first.

Preparations

Log in to a head node:

```
ssh <user>@orcd-login.mit.edu
```

Check available Apptainer versions in modules:

```
module av apptainer
```

Load an Apptainer module, for example:

```
module load apptainer/1.4.2
```

> **Note**
>
> Apptainer modules support both `apptainer` and `singularity` commands.

Download an image

Search for an image that provides your target application, for example, on Docker Hub. Here is an example for downloading a Docker image to support PyTorch:

```
singularity pull my-image.sif docker://bitnami/pytorch:latest
```

The `my-image.sif` is the name of the image. You can name it as you like.

> **Note**
>
> In Apptainer, the command `singularity` is a soft link to an executable named `apptainer`, so all `singularity` commands on this page can be replaced by the `apptainer` command. They work the same.

Run a program interactively

When the image is ready, launch a container based on the image and then run your application in the container. If you want to work interactively to test and debug code, it is convenient to log in to the container shell, for example:

```
$ singularity shell my-image.sif
Apptainer> python
Python 3.11.9 (main, May 13 2024, 16:49:42) [GCC 12.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import torch
>>> # Run your programs here.
```

Alternatively, execute a command in the container to run your programs:

```
singularity exec my-image.sif python my-code.py
```

Use the full path to the image file if it is not in the current directory.

The `python` here is installed in the container and has nothing to do with the `python` or `miniforge` modules that have been installed on the host. As the Python environment in the container provides all the packages you need, you don't need to install any Python packages and their dependencies. Now you can see the advantage of using a container.

Submit a batch job

When the tests are completed, you can submit a batch job to run your program in the background. Here is a typical batch job script:

job.sh

```
#!/bin/bash
#SBATCH -t 01:30:00                 # walltime = 1 hours and 30 minutes
#SBATCH -N 1                        # one node
#SBATCH -n 2                        # two CPU cores
#SBATCH -p mit_normal     # a partition with Rocky 8 nodes
```

```
module load apptainer/1.4.2   # load modules
singularity exec my-image.sif python my-code.py   # Run the program
```

The last line is a command to run a Python program using Singularity.

Submit the job script using `sbatch`:

```
sbatch job.sh
```

More on using Singularity

In many cases, GPUs are needed to accelerate programs. As the GPU driver is installed on the host, use the flag `--nv` to pass necessary GPU driver libraries into the container, so that the program can "see" the GPUs in the container.

From an interactive job with a GPU, check if GPUs are available in a container:

```
singularity exec --nv my-image.sif nvidia-smi
```

Here is an example to run Python programs on GPUs:

```
singularity exec --nv my-image.sif python my-code.py
```

By default, the home directory and the `/tmp` directory are bound to the container. If your programs read/write data files in other directories (e.g. `$HOME/orcd/scratch` or shared lab storage), bind the paths to the container using the flag `-B`:

```
singularity exec -B /path/to/data my-image.sif python my-code.py
```

In summary, a commonly used syntax to run a program with Singularity is the following,

```
singularity exec [--nv] [-B <path-to-data>] <image-name> <executable-name>
[source-code-name]
```

The terms in `<>` are required, while the terms in `[]` are optional, depending on use cases.

Here is an example job script to run a Python program with a GPU and data files saved in `$HOME/orcd/scratch` or `$HOME/orcd/pool` directories:

```
#!/bin/bash
#SBATCH -t 01:30:00         # walltime = 1 hours and 30 minutes
#SBATCH -N 1                # one node
#SBATCH -n 2                # two CPU cores
```

```
#SBATCH -G 1                # one GPU
#SBATCH -p mit_normal_gpu   # a partition with Rocky 8 nodes

module load apptainer/1.4.2   # load modules
singularity exec --nv -B $HOME/orcd/scratch,$HOME/orcd/pool my-image.sif python
my-code.py   # Run the program
```

Build Singularity images

In the previous section, it is assumed that all needed packages have been installed in the image. Users need to build a new image if some needed packages do not exist in the image.

To save work for the building process, search for an image providing the right OS and necessary dependencies to support your target application, then use it as a base image and build your target application on top of it.

The following is an example of building Python packages, such as PyTorch and Pandas, in a container image.

First, download a Docker image that provides the Ubuntu OS and has Python and PyTorch installed already:

```
singularity build --sandbox my-image  docker://bitnami/pytorch:latest
```

The command `build` here does not build anything yet, but just downloads the image and converts it to a new format. The flag `--sandbox` tells `build` to convert the image to the Sandbox format, which is convenient for installing packages interactively.

Now you can install additional packages in the base image. In many installation processes, it requires virtual root privilege, which is enabled by the `fakeroot` command here. `fakeroot` is installed as a dependency of the `apptainer/1.4.2` module.

Start a container shell with the flags `--writable --fakeroot`, which enables the write permission and virtual root privileges in the container, and then you can install your packages. Here is an example:

```
$ singularity shell --writable --fakeroot my-image
Apptainer> apt-get update
Apptainer> pip install pandas
Apptainer> python
Python 3.8.17 (default, Jun 16 2023, 21:48:21)
[GCC 10.2.1 20210110] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import torch
```

```
>>> import pandas as pd
```

Here, the `apt-get` is to install system software on an Ubuntu machine, which is supported by the `fakeroot` flag. The `pip install` is used to build Python packages. Finally, the package Pandas is built in the base image with PyTorch.

Alternatively, you can build a Docker image on a laptop/desktop, such as a Mac or PC, transfer it to the cluster, and run it with Singularity. Many packages have been developed and built in the Docker environment, so this approach is more convenient in many cases.

> **Note**
>
> Images that have been built on one architecture cannot be used on a computer that has a different architecture. For example, if you built an image on a laptop with an ARM architecture (e.g., a Mac laptop with an M1, M2, or M3 chip, etc.), you will not be able to run your container on Engaging, which runs on x86.

Definition files

It is sometimes advisable to use Apptainer definition files to document exactly how you built your image. In a definition file, you can specify installation commands, environment variables, files to create, and more. This method also avoids the need to use `--sandbox` and `--writable` (i.e., building images interactively), which is not compatible with some filesystems. Here is an example of a definition file:

my_image.def

```
Bootstrap: docker
From: rockylinux:8

%files
    $HOME/freesurfer-Rocky8-8.0.0-1.x86_64.rpm /root/freesurfer-Rocky8-8.0.0-
1.x86_64.rpm
    $HOME/license.txt /usr/local/freesurfer/8.0.0-1/.license

%post
    dnf -y upgrade dnf
    dnf -y upgrade rpm
    dnf -y install libgomp
    dnf -y install mesa-dri-drivers
    dnf install -y findutils which

    dnf -y --nogpgcheck localinstall /root/freesurfer-Rocky8-8.0.0-1.x86_64.rpm
    rm /root/freesurfer-Rocky8-8.0.0-1.x86_64.rpm
```

```
    dnf clean all

%environment
    export PATH=/usr/local/freesurfer/8.0.0-1/bin:/usr/local/freesurfer/8.0.0-
1/fsfast/bin:/usr/local/freesurfer/8.0.0-1/tktools:/usr/local/freesurfer/8.0.0-
1/mni/bin:$PATH
    export FREESURFER_HOME=/usr/local/freesurfer/8.0.0-1
```

In the above definition file, the `%files` section is where you list any files that you would like to copy over to the container environment. Here I have specified an `.rpm` file used for installing the Freesurfer software, as well as a license file. In the `%post` section, you specify the commands used to install your software and any dependencies, and the `%environment` section is used to set environment variables. At the top of the file, I have indicated that I want to use the Rocky 8 base image provided by Docker.

You can build this container using the following (preferably in a job on a compute node):

```
module load apptainer/1.4.2
apptainer build --fakeroot my_image.sif my_image.def
```

Tip

We have several container images and definition files saved on Engaging that you are free to use or look at for examples. You can find them here: `/orcd/software/community/001/container_images`

Further reference

For more information on building Apptainer/Singularity images, please refer to the official documentation here.

December 3, 2025